

THE AI TOOLBOX · CLASS 03

The AI Research Pipeline

See the whole research journey before walking a single step.

Merit Point Academy · 60 min · press **S** notes · → next ·  to comment

RECAP · WHERE WE LEFT OFF

Last class in 60 seconds

The question ladder

Vague → better → **strong**. A strong question names what you'll measure and what counts as success.

The recipe

Input + output + metric = an AI research question. If you can't picture the data table, it's not a question yet.

Generate, then verify

Funnel · SCAMPER · ChatGPT to generate;
Google Scholar · dataset check to verify.

Peer review (5 min): swap your three homework questions with a neighbor. Check their **strong** one: does it name input, output, metric? Would it pass the picture test? One sentence of feedback each.

Where we're going

- **0–8'** Peer review + the dataset that woke up AI
- **8–18'** The pipeline: Data → Model → Experiment → Conclusion
- **18–32'** The five steps, one by one — and where projects go wrong
- **32–40'** Data-driven thinking · supervised vs. unsupervised
- **40–46'** Tools: Google Colab · Kaggle
- **46–54'** Two labs: the whole pipeline in 25 lines · overfitting live
- **54–60'** Your pipeline diagram, homework & wrap-up
- Quizzes &  comments throughout

By the end: name the four stages of the pipeline, tell supervised from unsupervised, and place every later class of this course on the map you drew.

WARM-UP · ACTIVATE

Quick check · tap an answer

Between the question 'Can a car detect pedestrians at night?' and the result 'it finds 92% of them' sits a whole machine. What is that machine?

Luck — some models just work

A research pipeline: data → model → experiment → conclusion

A single genius programmer typing very fast

Buying a bigger computer

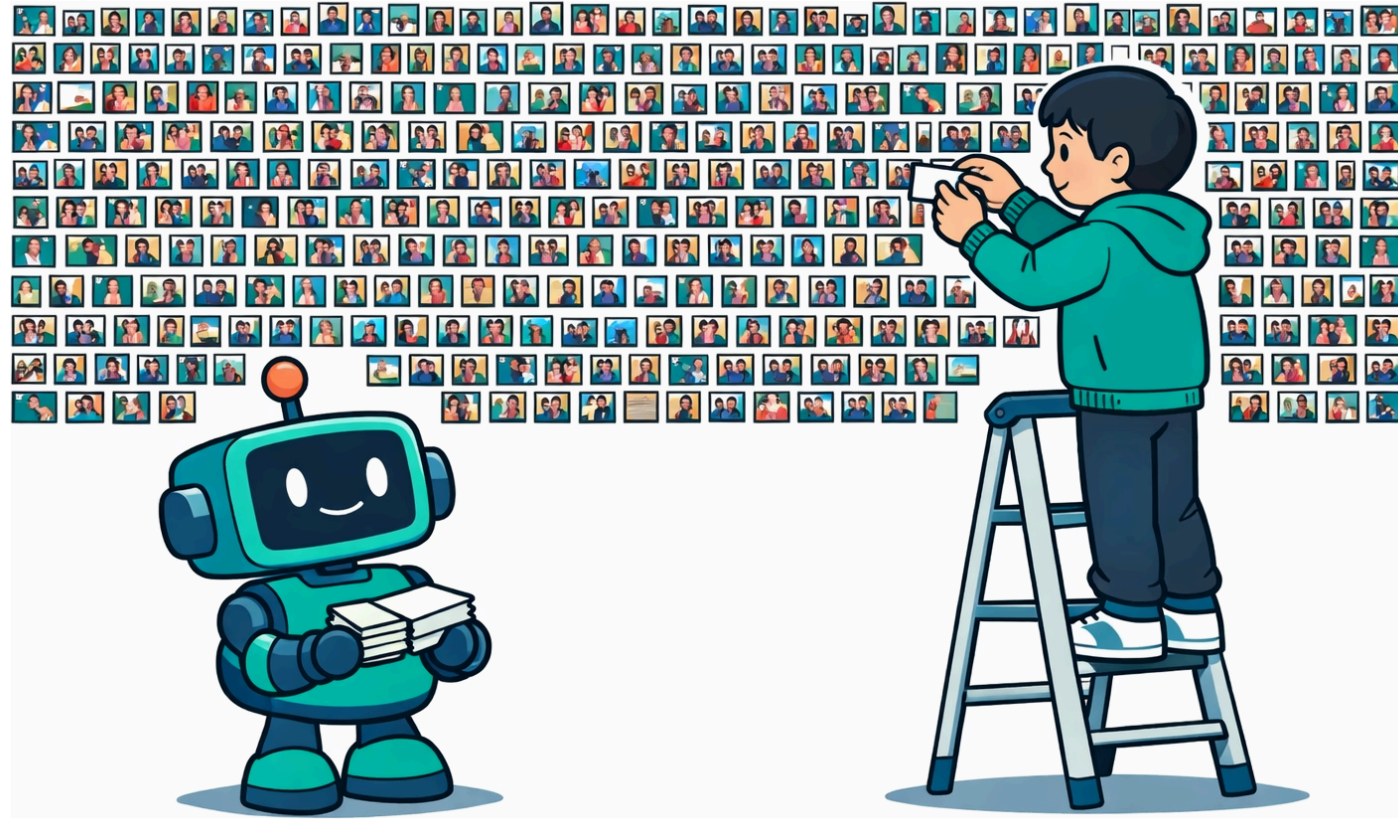
The dataset that woke up AI

For decades, researchers tweaked algorithms and stayed stuck. In 2009, **Fei-Fei Li** bet on the opposite idea: maybe the bottleneck was **data, not models**. Her team labeled **millions of images by hand** to build ImageNet. In 2012, a neural network trained on it crushed every competitor — and the deep-learning era began. The breakthrough wasn't a cleverer model. **It was the data stage of the pipeline.**



Speak this story

Neural voice · click to play / pause



Remember Class 1's history: the 2010s boom came from data + compute, not just cleverer math. ImageNet is that lesson with a name and a face.

CONCEPT 1 · THE SPINE OF THIS COURSE

Data → Model → Experiment → Conclusion

Every ML study — a student project or a billion-dollar lab — walks the same four stages:

1

Data

Examples to learn from — gathered,
cleaned, split.

2

Model

The thing that learns a pattern from the
examples.

3

Experiment

A fair measurement on data the model
hasn't seen.

4

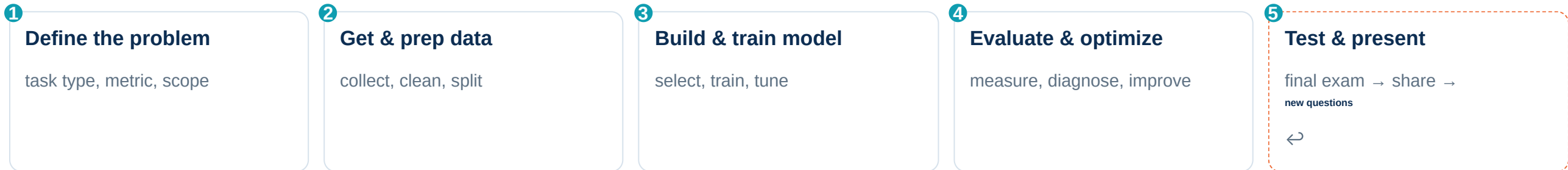
Conclusion

Only what the numbers support — no more.

KEY IDEA — every later class of this course is **one stage of this loop, in depth**: data (7–9), models (10–13), experiments (14), presentation (15). Learn the map today; walk it for twelve weeks.

Not a line — a cycle

Real projects run the loop with five named steps — and the arrow bends back:



Step 5's feedback **loops back to step 1**: presenting results surfaces new questions, and the cycle runs again. Research isn't a one-way road — it's laps around a track, each lap better than the last.

CHECK · THE LOOP

Quick check · tap an answer

Your model scores badly in the experiment stage. In a healthy research cycle, what happens next?

The project is dead — pick a new topic

Hide the bad number and present something else

Loop back: diagnose whether the problem is the data, the model, or the question — then iterate

Buy a GPU and hope

STEP 1 OF 5 · PROBLEM DEFINITION

Decide what game you're playing

Task type

Is the output a **category** (classification), a **number** (regression), or **groups with no labels** (clustering)? This one choice picks your models and metrics.

Success metric

Classification → accuracy, recall, F1.
Regression → error (MSE/MAE). Set the **target number** up front: "accuracy $\geq$ 95% = success."

Scope

What data can you actually get? What hardware and time do you have? Boundaries make the project **finishable**.

Sound familiar? This is Class 2's **input** → **output** → **metric** recipe, now wearing engineering clothes. Your strong question already did step 1 — that's why we wrote it first.

"Garbage in, garbage out"



Get the data

Public: Kaggle, UCI, Hugging Face — free, labeled, ready. **Your own:** sensors, forms, photos, scraping. Most student projects should start public — collection is a project in itself.



Prep the data

Clean: fix noise, fill or drop missing values. **Engineer features:** pick and transform the useful columns. **Split:** train / validation / test — three piles with three different jobs.

Why it's step 2 and not an afterthought: the model can only ever be as good as what it eats. ImageNet won 2012 at this step, not step 3 — and pitfall #1 (coming up) says spend **half your time here**.

STEP 3 OF 5 · MODEL CONSTRUCTION & TRAINING

Pick a learner, let it learn, tune the knobs



Select

Match the tool to the task: simple problems → linear regression, decision trees. Messy, complex data → random forests, neural networks. **Start simple** — always.



Train

Feed the training pile to the model. Through many iterations it **learns the mapping** from features to answers — exactly Class 1's "learn from data, not rules."



Tune

Models have knobs — learning rate, tree depth. Adjust, re-measure, repeat. Like tuning an instrument: small turns, listen each time.

Classes 10–13 live entirely inside this box: regression → trees → forests → neural nets, each one hands-on.

Diagnose like a doctor: two classic diseases



Underfitting — too simple

The model can't even learn the training data. **Symptom:** bad scores everywhere.

Treatment: a more capable model, or better features.



Overfitting — too complex

The model **memorized** the training data, noise and all. **Symptom:** great on training data, bad on new data.

Treatment: more data, regularization, or a simpler model.

The tell: always compare **training score vs. unseen-data score**. A big gap = overfitting. You'll see this live in Lab 2 — a model that scores perfectly on what it saw and falls apart on what it didn't.

The final exam — then tell the world



Final test

One last measurement on the **test set** — data the model has **never** seen, not even during tuning. This is the honest, unbiased answer to your research question. **Used once.**



Present

Paper, report, slides, or blog: problem → method → results → conclusion → future work. Class 2's tip: that "future work" section is where **someone else's next question** comes from — now you're writing one.

Golden rule: the test set is sacred. Tune on it even once and its score becomes a lie — that's "data leakage," pitfall #3. Train on the train pile, tune on the validation pile, touch the test pile **once**.

Quick check · tap an answer

A student's model scores 99% on the training data but only 62% on new data. What's the diagnosis, and one sensible treatment?

Underfitting — use a much bigger model

Overfitting — get more data, regularize, or simplify the model

Perfect health — report the 99%

The computer is broken

Rules in — or rules out?

Traditional programming

data + **rules** → answers

A human writes every rule by hand. The computer just executes. (Deep Blue, calculators, thermostats.)

Machine learning

data + **answers** → rules

Show examples with known answers; the machine **infers the rules itself** — including rules no human could write down.

This is why data is a pipeline stage at all: in ML, data isn't fuel for a program — data **is** the program's teacher. The essence of AI research: automatically discovering patterns in data that humans can't perceive.

Supervised vs. unsupervised



Supervised — with answer key

Every example comes **labeled**: this image is a pedestrian, this house sold for \$340k. The model learns to map input → label.

Classification and regression both live here.



Unsupervised — no answer key

No labels at all. The model finds **structure on its own**: clusters of similar customers, groups of similar songs.

Clustering is the classic example.

Our running example is supervised: pedestrian detection learns from boxes **humans drew** — that hand-labeling is exactly what made ImageNet valuable, and it's why the data stage costs real effort. Most of this course is supervised learning.

Quick check · tap an answer

You have 10,000 customer records and want to discover natural groups of similar shoppers — no predefined categories. Which kind of learning is this?

Supervised — the records are the labels

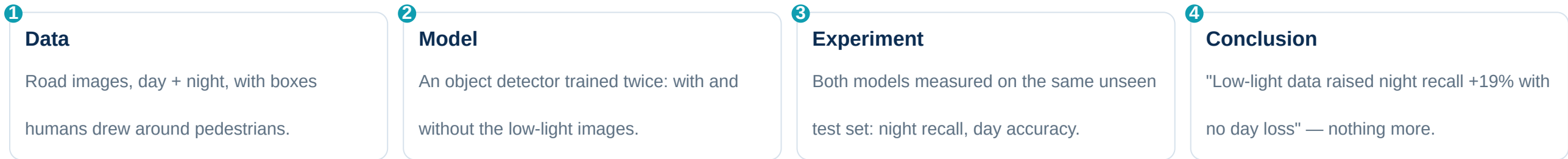
Unsupervised — finding structure without an answer key

Neither; this isn't machine learning

Both at once, always

Teaching a car to see, end to end

Take Class 2's strong question — "do low-light training images raise night recall without hurting day accuracy?" — and walk it through all four stages:



Deja vu? You already ran stages 3–4 of exactly this study — Lab 2 of Class 2 computed those recalls from toy data. Today's Lab 1 runs the *whole* loop. Prefer another domain? Same four stages, any field.

Predicting house prices — a regression lap

1–2 · Problem & data

Predict a **number** (price) → regression. Data: a classic housing dataset — hundreds of homes, features like rooms, age, neighborhood crime rate.

3–4 · Model & evaluate

Baseline first: **linear regression**. Then a **random forest** for the nonlinear patterns.
Metric: prediction error (MSE) — lower is better.
Tune, compare, iterate.

5 · Present

Report the error — and the **feature importances**: which factors actually drive price? That insight is often worth more than the prediction.

Notice what changed (task type, metric, models) and what didn't (**the loop**). Classification or regression, cars or houses — same pipeline. This exact project is your stretch homework on Kaggle.

AI ON SCREEN · TOOLBOX · YOUR CLOUD LAB

Google Colab — a free AI lab in your browser

FEATURE: ZERO SETUP · FREE GPU · SHARE LIKE A DOC

A free cloud notebook from Google: **Python pre-installed**, nothing to configure, runs on Google's machines — including **free GPUs** that train models many times faster than your laptop. Share a notebook like a Google Doc. This is where your real projects will live from Class 5 on.

▶ Watch the clip

CO
LAB



scan / tap to open

Kaggle — datasets, competitions, and other people's code

FEATURE: DATA + COMPETITIONS + LEARN FROM THE BEST

The world's largest data-science community. Three things you'll use: **datasets** (thousands, free, labeled — your step-2 shortcut), **competitions** (real problems, live leaderboards), and **notebooks** — top competitors publish their full code, so you can **read how experts run the pipeline**.

▶ Watch the clip

KAGGLE



scan / tap to open

The entire pipeline in 25 lines

Data → model → experiment → conclusion — **for real**, right here. We teach a model to name iris flowers from four measurements. Watch which comment each stage lives under.

```
1 # THE WHOLE RESEARCH PIPELINE - in your browser
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.tree import DecisionTreeClassifier
5
6 # --- 1. DATA: 150 real flowers, 4 measurements each, 3 species ---
7 X, y = load_iris(return_X_y=True)
8 names = ["setosa", "versicolor", "virginica"]
9
10 # split: train pile to learn from, test pile for the honest exam
11 X_tr, X_te, y_tr, y_te = train_test_split(
12     X, y, test_size=0.3, random_state=42)
```



Run



Explain each line



Explain



Debug



Improve



Mira

Python loads on first Run

► Run the code to see the output here.

Watch overfitting happen (train vs. test)

The same data, three models of growing complexity. Watch the **training error** fall while the **unseen-data error** turns around and climbs — that turn is overfitting, live.

```
1 import numpy as np
2
3 # The true relationship is a gentle curve + noise
4 rng = np.random.default_rng(7)
5 x = np.linspace(0, 1, 30)
6 y = np.sin(2 * np.pi * x) + rng.normal(0, 0.25, 30)
7
8 # three piles? two today: shuffle, then 20 points train / 10 test
9 idx = rng.permutation(30)
10 x_tr, y_tr = x[idx[:20]], y[idx[:20]]
11 x_te, y_te = x[idx[20:]], y[idx[20:]]
12
```



Run



Explain each line



Explain



Debug



Improve



Mira

Python loads on first Run

► Run the code to see the output here.

YOUR PROJECT

Draw your research pipeline

Turn **your** strong question from Class 2 into a diagram of the whole journey — so you always know which stage you're standing on.

- Draw the four stages, left to right: **Data** → **Model** → **Experiment** → **Conclusion**
- Under each stage, write what happens **for your question** — concretely
- Mark ⚠ where you predict the hard part will be

Deliverable

A labeled pipeline diagram for your chosen question.

Success looks like

All four stages, in order — and each names a **concrete artifact**: a dataset, a model, a number.

BUILD IT NOW · HANDS-ON

From diagram to mini research plan

1. Re-run **Lab 1** and name the four stages out loud from the code comments
2. Sketch your pipeline diagram — paper, slides, or whiteboard (photo it!)
3. Upgrade it to a **mini research plan** — one line per step: problem restated · target metric & threshold · data source · model path (baseline → better) · how you'll split & evaluate
4. Mark ▲ on the hardest stage, and write *one sentence* on why

Google Colab

Kaggle

ScholarMind

Tip: a plan that says "baseline first, fancy model second" is already smarter than half the internet. Steal that structure from the template on the next slide.

What a finished mini-plan looks like

Example question: *"Can a model tell cats from dogs in photos?"*

1 · Problem: binary image classification — photo in, cat/dog out.

2 · Success metric: accuracy $\geq$ 95% on unseen photos (F1 as backup).

3 · Data plan: Kaggle cats-vs-dogs dataset; resize all images; augment; split 70/10/20.

4 · Model path: baseline = small CNN → then fine-tune a pretrained ResNet50.

5 · Evaluation: tune on the validation pile only; touch the test pile once, at the end.

⚠ Predicted hard part: data — photos vary wildly in lighting and pose; augmentation will matter.

Every line is **concrete**: a named dataset, a number to beat, a model path from simple to strong, an evaluation discipline. Measure your plan against this bar.

Four classic pitfalls (and the fix)

1 · Worshipping models, ignoring data

Weeks on a fancy model fed garbage data. **Fix:** spend ~**half your time** on understanding and cleaning data. (ImageNet's whole lesson.)

2 · Starting with the fanciest model

Jumping straight to the deep end. **Fix:** baseline first — a simple model that sets the bar and reveals how hard the problem really is.

3 · Grading on the training data

"99% accuracy!" ...on data it memorized. **Fix:** the three piles. Report only unseen-data scores; touch the test set once.

4 · Numbers with no picture

A wall of metrics nobody understands. **Fix:** visualize — one honest chart beats a table of digits. (Class 9 is all about this.)

Today's vocabulary

Pipeline — the loop every study walks: data → model → experiment → conclusion.

Train / validation / test sets — three piles: learn from, tune on, final-exam once.

Underfitting / overfitting — too simple to learn vs. memorized the noise; diagnose via the train-vs-test gap.

Supervised / unsupervised — learning with labeled answers vs. finding structure without labels.

Baseline — the simple model you build first; everything fancier must beat it.

Data leakage — test information sneaking into training or tuning; makes scores lie.

RECOMMENDED HOMEWORK

Before next class

1. **Finish your pipeline diagram** + mini plan (photo or doc) and submit in Google Classroom — all four stages, each with a concrete artifact, ▲ on the hard part.
2. **Open Google Colab** tonight: new notebook, load any dataset (iris from Lab 1 is fine), print its shape and first rows. First lap, tiny scale — done is the goal.
3. **Re-run Lab 2** and find the degree where test error bottoms out — bring the number.

★ Stretch (optional)

Kaggle's "**House Prices — Advanced Regression Techniques**": skim the data page and one high-voted notebook. Spot the pipeline stages in someone else's real code.

[kaggle.com · House Prices](https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques)

"An ounce of practice..." — the Colab task is deliberately tiny. Starting is the assignment. Questions → 💬 on any slide.

TODAY'S LINE TO REMEMBER

"An ounce of practice is worth a pound of theory."

— Henry David Thoreau

You now hold the map. Next: walk it — one small lap tonight in Colab.



Speak this

WRAP-UP

Three things to keep

- One loop rules them all: **data** → **model** → **experiment** → **conclusion** — and it cycles
- Supervised = learning with an answer key; the answer key is **expensive and precious**
- The test set is sacred: three piles, honest numbers, baseline first

Exit ticket: which pipeline stage do you predict will be hardest for *your* project, and why? Post it in .

Next · Class 4 — ChatGPT for research: put an AI assistant to work on every stage of this pipeline.

BACKUP MATERIAL

Extra slides — if there's time

Deeper dives and extra practice. Skip in a tight hour; pull up on demand or for a longer session.

Research ethics

privacy, bias, impact

The three piles, illustrated

train / validation / test

Bonus quiz

one more check

The pipeline has a conscience: research ethics



Data privacy

Is your data legally sourced and properly authorized? Faces, health records, chat logs — being *available* doesn't make them *fair game*. (A data-stage question.)



Algorithmic bias

If the training data under-represents a group, the model fails them — a detector trained mostly on daytime images misses at night. (A model-stage question.)



Social impact

Could your results be misused, or widen inequality? Anticipating harm is part of the conclusion stage — not someone else's job.

"As researchers we are not only creators of technology but bearers of responsibility." Notice each concern maps to a pipeline stage — ethics isn't a separate box, it rides **inside every box**.

Three piles, three jobs



Training set (~70%)

The textbook. The model **learns** from this pile — it may read it as many times as it likes.



Validation set (~10%)

The practice quiz. You **tune knobs** against this pile — compare models, pick settings.



Test set (~20%)

The final exam, sealed until the end. Opened **once**. Its score is the number you report.

Why so strict? Anything you optimize against, you slowly memorize — a student who practices on the final exam hasn't learned, just leaked. Same for models: that's **data leakage**.

Quick check · tap an answer

A researcher tries 50 different model settings, each time checking the score on the test set, and reports the best one. What went wrong?

Nothing — trying many settings is good practice

They used data leakage: tuning against the test set means its score no longer measures unseen data

They should have tried 500 settings

Models may only be tuned once by law